Kirana-Register Backend

Database Schema

MongoDB Collections

1. **users**

Field	Type	Notes
`_id`	ObjectId	Auto-generated
`username`	String	Unique
`password_hash`	String	BCrypt hashed
`store_ids`	List<String> / null	References stores; null for admins
`roles`	List<String>	USER,ADMIN
`store_roles`	List<{Store_id,Role}>	ex: [“jfdklajljfdla”: STORE_OWNER]
`is_active`	Boolean	Default true
`created_at`	Timestamp	
`updated_at`	Timestamp	

> The reason List of pairs is created here is because , the previous design was based on RDB’s and since mongo is inefficient in joins , this model is adapted as it can query all the roles in one call.

2. **stores**

Field	Type	Notes
`_id`	ObjectId	
`store_name`	String	UNIQUE (or should use store_id in case oof common name)
`region`	String	
`address`	String	
`base_currency`	String	INR, USD, EUR .., this is the currency used in this store.
`created_at`	Timestamp	
`updated_at`	Timestamp	when stores details are changed

3. **products**

Field	Type	Notes
`_id`	ObjectId	
`product_name`	String	
`category`	String	
`display_price`	Decimal	Always in the store's base currency.
`is_active`	Boolean	
`created_at`	Timestamp	
`updated_at`	Timestamp	when some details are changed, like price and stuff.

4.

PostgreSQL Tables

4. **transactions**

Field	Type	Notes
`_id`	UUID	
`user_id`	String	Who made this transaction
`transaction_type`	String	`CREDIT` / `DEBIT`
`payment_currency`	String	The currency the **user chose to pay in**
`total_amount`	Decimal	Total in `payment_currency`
`created_at`	Timestamp	

> This table only cares about the user's side i.e.., what they paid and in what currency.

5. **transaction_items**

Column	Type	Constraint	Notes
`id`	INTEGER	PK	

`transaction_id`	VARCHAR	NOT NULL	Ref to transactions table
`product_id`	VARCHAR	NOT NULL	Ref to MongoDB products
`store_id`	VARCHAR	NOT NULL	Ref to MongoDB stores
`quantity`	INTEGER	NOT NULL	
`unit_price_base`	NUMERIC(12,2)	NOT NULL	Product price in store's base currency
`base_currency`	VARCHAR(3)	NOT NULL	Store's base currency (denormalized for query convenience)
`conversion_rate`	NUMERIC(12,6)	NOT NULL	Rate: 1 unit of base_currency = ? payment_currency
`unit_price_payment`	NUMERIC(12,2)	NOT NULL	`unit_price_base × conversion_rate`
`total_base`	NUMERIC(15,2)	NOT NULL	`quantity × unit_price_base`
`total_payment`	NUMERIC(15,2)	NOT NULL	`quantity × unit_price_payment`
`created_at`	TIMESTAMP	DEFAULT NOW()	

> This is the detailed line-item table. It captures the conversion math so i can always audit how the amounts were derived.

6. **store_transactions**

Column	Type	Constraint	Notes
`id`	UUID	PK	
`store_id`	VARCHAR	NOT NULL	Ref to MongoDB stores
`transaction_id`	VARCHAR	NOT NULL	Ref to transactions
`transaction_type`	VARCHAR(10)	NOT NULL	CREDIT / DEBIT
`total_in_base_currency`	NUMERIC(15,2)	NOT NULL	Sum of all `total_base` for this store in this transaction
`created_at`	TIMESTAMP	DEFAULT NOW()	
`status`	VARCHAR(15)		PROCESSING, COMPLETED
`base_currency`	VARCHAR(3)	NOT NULL	Store's base currency Note: This field is used when , let's say that a store owner who has multiple stores which in turn have different base currencies , wants to know total sales that happened in a particular base currency.

> To generate a monthly report for a store, I will just write the below query:
>
> ```
> SELECT
>   SUM(CASE WHEN transaction_type = 'CREDIT' THEN total_in_base_currency ELSE 0 END) as total_credit,
>   SUM(CASE WHEN transaction_type = 'DEBIT' THEN total_in_base_currency ELSE 0 END) as total_debit
> FROM store_transactions
> WHERE store_id = ? AND created_at BETWEEN ? AND ?;
>
> ```

7. **inventory**

Column	Type	Constraint	Notes
`id`	INTEGER	PK	
`store_id`	VARCHAR	NOT NULL	Ref to MongoDB stores
`product_id`	VARCHAR	NOT NULL	Ref to MongoDB products
`quantity`	INTEGER	NOT NULL	Current stock
`created_at`	TIMESTAMP	DEFAULT NOW()	
`updated_at`	TIMESTAMP	DEFAULT NOW()	

8. **refunds**

Column	Type	Constraint	Notes
`id`	UUID	PK	
`transaction_id`	VARCHAR	NOT NULL	Original transaction id
`store_id`	VARCHAR	FK → stores in MongoDB	For store_owners/store_admins to quickly check their refund requests.
`refund_amount_payment`	NUMERIC(15,2)	NOT NULL	Amount refunded in user's payment currency

`payment_currency`	VARCHAR(3)	NOT NULL	
`reason`	TEXT		
`status`	VARCHAR(15)	NOT NULL	`PENDING`, `COMPLETED`, `REJECTED`
`created_at`	TIMESTAMP	DEFAULT NOW()	

9. **refund_items**

Column	Type	Constraint	Notes
`id`	UUID	PK	
`refund_id`	UUID	FK → refunds.id	
`product_id`	INTEGER	FK → product.id	
`quantity_refunded`	INTEGER	NOT NULL	
`refund_amount_base`	NUMERIC(12,2)	NOT NULL	In store's base currency
`refund_amount_payment`	NUMERIC(12,2)	NOT NULL	In user's payment currency
`created_at`	TIMESTAMP	DEFAULT NOW()	

Logic behind Kirana-Backend functionalities

1. Currency Flow/ Logic Implementation in a normal transaction :

Say a user pays in **USD** and buys from two stores:

1. **Store A** (base: INR) — Rice Rs. 40 × 2 = Rs. 80
2. **Store B** (base: EUR) — Olive Oil Er. 5 × 1 = Er. 5

With rates: 1 INR = 0.012 USD, 1 EUR = 1.08 USD

transaction (PostgreSQL): `payment_currency: USD, total_amount: 6.36` (0.96 + 5.40)

transaction_items (PostgreSQL): two rows showing the per-item conversion math

store_transactions (PostgreSQL): two rows:

- Store A: `total_in_base_currency: 80 INR, type: CREDIT`
- Store B: `total_in_base_currency: 5 EUR, type: CREDIT`

Reports for Store A just query `store_transactions WHERE store_id = 'A'`

---

2. Refund Example

Using the same transaction from before:

Original Transaction (TXN_1001):

- Store A (base: INR) — Rice ₹40 × 2 = ₹80
- Store B (base: EUR) — Olive Oil €5 × 1 = €5
- User paid in USD → total $6.36

Now the user wants to return 1 Rice from Store A.

Stage 1: Refund Requested (PENDING)

Lookup original `transaction_items` for Rice:

- `unit_price_base` = ₹40
- `conversion_rate` = 0.012
- `unit_price_payment` = $0.48

Calculate:

- `refund_amount_base` = 1 × ₹40 = **₹40**
- `refund_amount_payment` = 1 × $0.48 = **$0.48**

refunds (PostgreSQL):

id	transaction_id	store_id	refund_amount_payment	payment_currency	status
REF_01	TXN_1001	Store A	0.48	USD	`PENDING`

refund_items (PostgreSQL):

refund_id	transaction_item_id	quantity_refunded	refund_amount_base	refund_amount_payment
REF_01	TI_001 (Rice)	1	40.00 INR	0.48 USD

store_transactions: No change yet | **inventory:** No change yet

Stage 2: Store Owner/Store Admin Approves

store_transactions (PostgreSQL) — new DEBIT row added:

store_id	transaction_id	transaction_type	total_in_base_currency	base_currency

Store A	TXN_1001	CREDIT	80.00	INR
Store B	TXN_1001	CREDIT	5.00	EUR
Store A	TXN_1001	**DEBIT**	**40.00**	INR

inventory: Rice quantity in Store A → **+1**

refunds: `status → COMPLETED`

Report Query for Store A

```
SELECT
  SUM(CASE WHEN transaction_type = 'CREDIT' THEN total_in_base_currency
ELSE 0 END),
  SUM(CASE WHEN transaction_type = 'DEBIT' THEN total_in_base_currency
ELSE 0 END)
FROM store_transactions
WHERE store_id = 'Store A';

```

	Amount
Total Credit	₹80
Total Debit	₹40
Net Flow	**₹40**

Store A originally received ₹80, gave back ₹40 for the refund , thus net = ₹40. Store B's report is completely unaffected since the refund had nothing to do with it.

Kafka

reports table (MongoDB)

Column	Type	Constraint	Notes
id	UUID	PK	
store_id	VARCHAR	NOT NULL	Ref to MongoDB stores
report_type	VARCHAR	NOT NULL	WEEKLY, MONTHLY, YEARLY
period_start	DATE	NOT NULL	Start of reporting period
period_end	DATE	NOT NULL	End of reporting period
base_currency	VARCHAR(3)	NOT NULL	Store's base currency
total_credits	NUMERIC(15,2)		Filled after processing

total_debits	NUMERIC(15,2)		Filled after processing
net_flow	NUMERIC(15,2)		credits – debits
created_at	TIMESTAMP	DEFAULT NOW()	When the request was made

```
Admin/Owner sends a generate report  req to server
→ server replies with PROCESSING successful to store_transactions table
→ adds job to Kafka's queue
→ job gets processed and send report in MongoDB.
→ Store_transaction's status for that particular transaction is set to "completed"
```

Redis

1. Currency Rate Caching

The FX API has rate limits and the rates don't change every second. So i cache them.

Format:

```
Key:   "fx_rate:INR:USD"
Value: "0.012"
TTL:   50 minutes
```

> The reason for 50 minutes is because , the free plan in FX-Rates API will only provide hourly update and allows only 1k requests per month.
>
> TTL of
>
> ```
> 15 min → 4 req/hour → 2.8k req/month → invalid
> 30 min → 2 req/hour → 1.4k req/month → invalid
> 40 min → 4 req/hour → 1.080k req/month → invalid
> 50 min → 28.8 req/day → ~850 req/month → valid
> ```

2.

2. Rate Limiting

Format:

```
rate_limit:<clientId>:count
rate_limit:<clientId>:lastRefill
```

> Algorithm chosen : Token Bucket algorithm
>
> Reason: Simple, Memory efficient

3. Distributed locking

Format:

3.1 To prevent overselling

```
Key:   "lock:<inventory>:product_id"
```

Will be using Redisson

> Scenario:
>
> 1. I will be generating a lock to lock a particular product during a transaction so that no 2 users can perform transaction at the same time.
> 2. Sine I will be going through a list of items , it will be easy for me to store a list of keys of above format where I can generate a lock with multilock function from remission.
> 3. Since lua scripts ensure atomicity by locking the redis db thus preventing all other actions from happening, I need to run the locking loop logic for all the products in a single go .There might come a situation where I would need to lock a product which is locked by some other transaction. In that case, I need to implement a retrying logic so that this Lua script again tries to go through that array of keys to generate locks that too after deleting all the locks that I have acquired before in that same transaction since partial transactions are much worse.
> 4. Generating locks one by one for all the products present in a transaction will also not work which would mean that in a complete transaction, some of the items were successfully sold while some others weren't since there came a product req that was locked by someone else at that same time thus again resulting in partial transaction of a user.
> 5. In the case of Redisson, what happens is that , when I try to create a lock on a list of product_id (products), it actually forces the other person to wait for some time till this current lock is resolved , and once resolved, the other person can actually continue from where he was waiting.

Flows

1. Auth Flow

```
Client → JWT Filter → Spring Security Context → Controller → Service

```

2. Store APIs

```
Admin(JWT) → StoreController → StoreService → StoreDao → StoreRepo →
MongoDB

```

3. Product APIs

```
Client(JWT) → ProductController → ProductService → StoreDao(validate
store + match currency) → ProductDao → ProductRepo → MongoDB

```

4. Transaction Creation

```
Client(JWT) → TransactionController → TransactionService
  → ProductDao(MongoDB — fetch product details + store currency)
  → CurrencyService(FX API + Redis cache)
  → TransactionItemDao(JPA → PostgreSQL — save line items)
  → StoreTransactionDao(JPA → PostgreSQL — save per-store totals)
  → InventoryDao(JPA → PostgreSQL — decrement stock)
  → TransactionDao(TransactionRepo → PostgreSQL — save transaction)
  → Kafka Producer(TransactionCreated event)

```

5. Refund Request (User)

```
Client(JWT) → RefundController → RefundService
  → TransactionItemDao(PostgreSQL — lookup original items by transaction_id + product_id)
  → Currency(from PostgreSQL)
  → RefundDao(JPA → PostgreSQL — save refund as PENDING)
  → RefundItemDao(JPA → PostgreSQL — save refund items)

```

6. Refund Approval (Store Owner / Admin)

```
StoreOwner(JWT) → RefundController → RefundService
  → RefundDao(PostgreSQL — validate status is PENDING)
  → StoreTransactionDao(PostgreSQL — insert DEBIT row)
  → InventoryDao(PostgreSQL — increment stock)
  → RefundDao(PostgreSQL — update status to COMPLETED)

```

7. Report API

```
Client(JWT) → ReportController → ReportService → StoreTransactionDao → StoreTransactionRepo → PostgreSQL

```

8. Kafka Consumer (Async Report Generation)

```
Kafka Consumer(TransactionCreated) → ReportService → ReportsDao → ReportsRepo → MongoDB
→ send email

```

9. Monitoring

```
SpringBoot → Actuator → Prometheus → Grafana

```

10. Rate Limiting

```
Client → Rate Limit Filter(Redis — bucket token) → Controller

```

-

Api's

Base URL: `http://localhost:8080/`

All protected endpoints require: `Authorization: Bearer <JWT_TOKEN>`

1. Authentication

1.1 Register

`POST /auth/register`

Request :

```
{
  "username": "rahul_shop",
  "password": "StrongPassword"
}

```

Response :

```
{
  "message": "User registered successfully",
  "userId": "665a1b2c3d4e5f6a7b8c9d0e",
  "username": "rahul_shop",
  "role": [
      USER
    ],
  "createdAt": "2025-06-01T10:30:00Z"
  }
```

```
}
```

1.2 Login

`POST /auth/login`

Request :

```
{
  "username": "rahul_shop",
  "password": "Str0ng@Pass1"
}
```

Response :

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresInSeconds": 3600
}
```

1.3 Assign Role (ADMIN)

`POST /admin/assig-new-admin` — `ADMIN` only

Request :

```
{
  "username": "admin2",
  "password": "admin2"
}
```

Response :

```
{
  "message": "Role assigned successfully",
  "username": "admin2",
  "role" : {
     "ADMIN"
  }
}
```

1.4 Create a STORE_OWNER (ADMIN)

Only the admin can create a store_owner for the first time.

`POST /auth/register/store-owner`

Request :

```
{
  "username": "Rahul",
  "email": "rahul@gmail.com",
  "password": "Rahul"
   "stores": [
    {
      "storeName": "Rahul General Store",
      "region": "Bengaluru",
      "address": "No. 42, 5th Cross, Jayanagar, Bengaluru 560041",
      "baseCurrency": "INR"
    }
  ]
}
```

Response :

```
{
  "message": "Store Owner Created Successfully",
  "username": "Rahul",
  "password": "Rahul"
```

```
  "role" : {
     "USER"
  }
  "store_roles" : {
        {
          "665b2c3d4e5f6a7b8c9d0f02",
          "STORE_OWNER"
        }
  }
}

```

1.5 Change password

After the admin has passed out his credentials to store owner, he should change the pwrd to something of his own choice. Basic auth to be used at first.

`POST /auth/creds`

Request :

```
{
  "password": "Rahul new pwrd"
}

```

Response :

```
{
  "Password changed successfully"
}

```

---

2. Stores

2.1 Create Store

`POST /stores` — `STORE_OWNER/STORE_MANAGER` only

Request :

```
{
  "storeName": "Rahul General Store 2 ",
  "region": "Bengaluru",
  "address": "No. 43, 6th Cross, Jayanagar, Bengaluru 560041",
  "baseCurrency": "INR"
}

```

Response :

```
{
  "success": true,
  "storeId": "665b2c3d4e5f6a7b8c9d0f01",
  "storeName": "Rahul General Store 2 ",
  "region": "Bengaluru",
  "address": "No. 43, 6th Cross, Jayanagar, Bengaluru 560041",
  "baseCurrency": "INR",
  "createdAt": "--------",
  "updatedAt": "--------"
}

```

2.2 Get Store

`GET /stores/` — Owner/Admin

Response :

```
{
  "success": true,
  "data": {
            {
    "storeId": "665b2c3d4e5f6a7b8c9d0f02",
    "storeName": "Rahul General Store",
    "region": "Bengaluru",
    "address": "No. 42, 5th Cross, Jayanagar, Bengaluru 560041",
    "baseCurrency": "INR",
            },
            {
    "storeId": "665b2c3d4e5f6a7b8c9d0f01",
    "storeName": "Rahul General Store 2 ",
    "region": "Bengaluru",
    "address": "No. 43, 6th Cross, Jayanagar, Bengaluru 560041",
    "baseCurrency": "INR",
            }
  }
}

```


---

3. Products

3.1 Create Product

`POST /stores/{storeId}/products` — Owner/Admin

Request :


```
{
  "productName": "Basmati Rice 5kg",
  "category": "Groceries",
  "displayPrice": 450.00,
  "quantity": 100
}

```


Response :


```
{
  "success": true,
  "productId": "665c3d4e5f6a7b8c9d0e1f02",
  "storeId": "665b2c3d4e5f6a7b8c9d0f01",
  "productName": "Basmati Rice 5kg",
  "category": "Groceries",
  "displayPrice": 450.00,
  "storesCurrency": "INR",
}

```


3.2 Get Products for a Store

`GET /stores/{storeId}/products?page=0` — Any authenticated

Response :


```
{
   [
      {
        "productId": "665c3d4e5f6a7b8c9d0e1f02",
        "productName": "Basmati Rice 5kg",
        "category": "Groceries",
        "displayPrice": 450.00,
        "storesCurrency": "INR",
        "isActive": true,
        "quantity": 100
      },
      {
        "productId": "665c3d4e5f6a7b8c9d0e1f02",
        "productName": "Basmati Rice 5kg",
        "category": "Groceries",
        "displayPrice": 450.00,
        "storesCurrency": "INR",
        "isActive": true,
        "currentStock": 100
      }
    ]
}

```

3.3 Update Product

`PUT /stores/{storeId}/products/{productId}` — Store Owner/Admin

Request :

```
{
  "productName": "Premium Basmati Rice 5kg",
  "displayPrice": 499.00,
  "category": "Groceries",
}

```

Response :

```
{
  "success": true,
  "productId": "665c3d4e5f6a7b8c9d0e1f02",
  "storeId": "665b2c3d4e5f6a7b8c9d0f01",
  "productName": "Premium Basmati Rice 5kg",
  "displayPrice": 499.00,
  "storesCurrency": "INR",
  "isActive": true,
  "currentStock": 100,
  "updatedAt": "2025-06-02T14:00:00Z"
}

```

---

4. Transactions (Rate Limited: 10 req/min)

4.1 Create Transaction

`POST /transactions` — Any authenticated

Request :

```
{
  "paymentCurrency": "USD",
  "transactionType": "CREDIT",
  "items": [
    {
      "productId": "665c3d4e5f6a7b8c9d0e1f02",
      "storeId": "665b2c3d4e5f6a7b8c9d0f01",
      "quantity": 2
    },
    {
      "productId": "665c3d4e5f6a7b8c9d0e1f10",
      "storeId": "665b2c3d4e5f6a7b8c9d0f02",
      "quantity": 1
    }
  ]
}

```

Response :

```
{
  "success": true,
  "transactionId": "665d4e5f6a7b8c9d0e1f0203",
  "userId": "665a1b2c3d4e5f6a7b8c9d0e",
  "transactionType": "CREDIT",
  "paymentCurrency": "USD",
  "totalAmount": 6.36,
}

```

---

5. Refunds

5.1 Request Refund

`POST /refunds` — Transaction owner

Request

```
{
```

```
  "transactionId": "665d4e5f6a7b8c9d0e1f0203",
  "reason": "Rice bag was damaged",
  "items": [
    {
      "productId": "665c3d4e5f6a7b8c9d0e1f02",
      "quantityRefunded": 1
    }
  ]
}
```

Response

```
{
  "success": true,
  "refundId": "d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f90",
  "transactionId": "665d4e5f6a7b8c9d0e1f0203",
  "refundAmountPayment": 0.48,
  "paymentCurrency": "USD",
  "reason": "Rice bag was damaged",
  "status": "PENDING",
  "items": [
    {
        "productId": "665c3d4e5f6a7b8c9d0e1f02",
        "productName": "Basmati Rice 5kg",
        "quantityRefunded": 1,
        "refundAmountBase": 40.00,
        "refundAmountPayment": 0.48
    }
  ],
  "createdAt": "2025-06-04T09:00:00Z"
}

```

5.2 Approve / Reject Refund

`PATCH /refunds/{refundId}/status` — Owner/Admin

On approval: inserts DEBIT into `store_transactions`, increments inventory, marks COMPLETED.

Request :

```
{
  "status": "COMPLETED"
}

```

Response :

```
{
  "refundId": "d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f90",
  "transactionId": "665d4e5f6a7b8c9d0e1f0203",
  "storeId": "665b2c3d4e5f6a7b8c9d0f01",
  "refundAmountPayment": 0.48,
  "paymentCurrency": "USD",
  "status": "COMPLETED"
}

```

---

6. Reports (Kafka — Async)

Logic:

As soon as my transaction happens, I will store the individual revenues of that store in store_transaction table whose initial status is processing. Once if Kafka has processed and generated the reports and saved it in mongoDB , I will set that transactions status as processed.

6.1 Get Report

`GET /reports/{store-id}/monthly` — Owner/Admin

> I will check wether there exists a column that still has its status pending for its store , if it does I will reply return inconsistent, else I will generate the report.

Response :


```
{
    "reportId": "RPT-20250630-ABCD1234",
    "storeId": "665b2c3d4e5f6a7b8c9d0f01",
    "reportType": "MONTHLY",
    "periodStart": "2025-06-01",
    "periodEnd": "2025-06-30",
    "baseCurrency": "INR",
    "totalCredits": 24500.00,
    "totalDebits": 3200.00,
    "netFlow": 21300.00,
    "status": "COMPLETED",
    "createdAt": "2025-07-01T08:00:00Z",
    "completedAt": "2025-07-01T08:00:12Z"
}
```